

POURUSH KASHYAP

Deep RAG Knowledge Base — Portfolio, Projects, Architecture & Contact

Purpose. This document is the detailed grounding source for Pourush Kashyap's personal RAG portfolio assistant. It is designed to answer recruiter, developer and visitor questions about Pourush's background, technical capabilities, projects, system architecture, implementation approach, current focus, roadmap and contact information.

Name	Pourush Kashyap
Professional positioning	AI & Agentic AI Developer / AI-ML Engineer in the making
Degree	B.Tech — Computer Science & Engineering, CT University
Batch	CSE '27
CGPA	9.2
Primary focus	AI/ML, Deep Learning, Generative AI, RAG, Agentic AI, AI-powered software systems
Portfolio style	AI systems portfolio with project case studies, skills, journey, background and contact sections

Critical correction for the RAG assistant

The Solitair Infosys internship project is Crime Scene Detection. The older portfolio data contained a stale Music Genre Classification entry, but the corrected knowledge source explicitly identifies Crime Scene Detection as the internship project. The assistant must use Crime Scene Detection and must not describe Music Genre Classification as the internship project.

Source principle

This knowledge base is grounded in the uploaded portfolio source, project case-study data and corrected portfolio/resume knowledge. Where source versions disagree, the RAG assistant should preserve the distinction rather than inventing a reconciliation. Future features are always labelled as future/planned.

1. Contact & Professional Identity

These details may be returned directly when a user asks how to contact Pourush, find his profiles, or request his professional links.

Full name	Pourush Kashyap
Email	pourushkashyap06@gmail.com
Phone	+91 79863 55170
GitHub	https://github.com/Pourushkashyap
LinkedIn	https://www.linkedin.com/in/pourush-kashyap-68890a289/
LeetCode	https://leetcode.com/u/_pourush2005/
Country	India
Availability	Open to AI/ML opportunities
Portfolio positioning	AI & Agentic AI Developer
Professional role	AI & Agentic AI Engineer

When to provide contact details

- If someone asks for Pourush's email, provide the email directly.
- If someone asks for LinkedIn, GitHub or LeetCode, provide the corresponding profile link.
- If someone asks how to contact him for an internship or project, provide email and optionally LinkedIn.
- Do not invent additional social profiles or contact channels.

Professional introduction

Pourush is a CSE student building intelligent systems that turn complex problems into production-ready AI applications. His portfolio emphasizes AI agents, RAG, LangGraph, MCP, Machine Learning, Deep Learning and full-stack engineering. The portfolio describes a progression from full-stack development to ML, deep learning, Generative AI, RAG, Agentic AI and autonomous AI systems.

2. Education, Background & Professional Direction

B.Tech	Computer Science & Engineering — CT University
Academic batch	CSE '27
CGPA	9.2
Class XII	Punjab Agricultural University — 92.4%
Class X	Holy Child Foundation High School — 98%
Academic focus	DSA, Machine Learning, Deep Learning, DBMS, Operating Systems, Software Engineering

Development philosophy

The portfolio describes a system-oriented approach rather than treating AI as an isolated model. The stated process is: Problem → Research → Architecture → Implementation → Evaluation → Testing → Deployment. The goal is to build complete systems where AI components, APIs, data, interfaces, testing and deployment work together.

How Pourush builds systems

- 1 Problem — understand user requirements, constraints and success criteria.
- 2 Research — study approaches, available data and system constraints.
- 3 Architecture — define components, data flow, agent responsibilities and interfaces.
- 4 Implementation — build the AI and software components together.
- 5 Evaluation — measure quality instead of assuming the model or agent works.
- 6 Testing — verify components, integrations, edge cases and regressions.
- 7 Deployment — move the system toward something people can actually use.

3. Technical Skill Map — Detailed

Programming	Python, C++, JavaScript, TypeScript
Frontend	React, Tailwind CSS, Vite, Zustand, Redux Toolkit, Axios
Backend	FastAPI, Flask, Node.js, Express.js, REST APIs, authentication, API integration
Databases	MongoDB, PostgreSQL, pgvector, MySQL
ML	Scikit-learn, regression, classification, preprocessing, feature engineering, model evaluation
Deep Learning	TensorFlow, Neural Networks, CNN, RNN, LSTM, Bidirectional LSTM, model training
Computer Vision	YOLOv8, object detection, image classification
GenAI	LLMs, Gemini, prompt engineering, embeddings, Hugging Face
RAG	FAISS, Chroma, embeddings, vector search, retrieval, context compression, document processing
Agents	LangChain, LangGraph, MCP, tool calling, memory, planning, multi-agent systems, state management
AI Engineering	Structured outputs, context management, error handling, agent evaluation, LLM evaluation
Deployment	Docker, Docker Compose, Dockerfile, CI/CD, cloud deployment, environment configuration
Engineering tools	Git, GitHub, VS Code, Postman, debugging, testing, API design, system design

Core contribution areas

- Build AI agents and multi-step workflows using LangGraph, LangChain and MCP.
- Build RAG applications using document processing, embeddings, vector search and retrieval.
- Integrate AI models and agents into React/FastAPI/Node.js applications.
- Build ML solutions using preprocessing, feature engineering, training, evaluation and inference.
- Package and deploy AI applications using Docker and modern development workflows.

Proficiency framing from the portfolio

Core	Python, C++, React, JavaScript, LangGraph, LangChain, RAG, MongoDB
Working knowledge	TensorFlow, FastAPI, Node.js, Docker, FAISS, Chroma, PostgreSQL
Exploring	MCP, AI Evaluation, Multi-Agent Systems, Production AI

4. Development Journey — From Full Stack to Autonomous AI

2023	JavaScript, React, Node, MongoDB
2024	Python, ML, Pandas, Scikit-learn
2025	TensorFlow, Deep Learning, LLMs, RAG
2026	LangGraph, MCP, Agents, Multi-Agent Systems, Docker

Stage 1 — Full Stack Development

Learned how frontend, backend, databases and APIs work together to create complete applications. This established the software-engineering foundation used by later AI projects.

Stage 2 — Machine Learning

Moved from deterministic software toward systems that learn patterns from data. Focus areas included preprocessing, feature engineering, model selection, training and evaluation. FitGenius AI is associated with this stage.

Stage 3 — Deep Learning

Explored neural networks, CNNs, RNNs and LSTMs and how models learn complex representations. Crime Scene Detection and SilentSOS are associated with this stage.

Stage 4 — Generative AI

Shifted toward LLM applications, prompt engineering, embeddings and natural-language interfaces.

Stage 5 — RAG

Started building systems where language models are grounded by external context. Portfolio flow: Documents → Chunking → Embeddings → Vector Store → Retriever → LLM → Grounded Answer.

Stage 6 — Agentic AI

Explored tool-using agents, LangGraph, memory, MCP, state and multi-agent workflows. The focus moved from one-shot LLM responses to systems that can reason over tasks and coordinate multiple steps.

Stage 7 — Autonomous AI Systems

Current direction: systems that can plan, execute, evaluate and recover from failures. AgentForge and Self-Healing Debugger represent this direction.

5. Project — Crime Scene Detection

Correct internship project at Solitair Infosys. Image-based crime-scene analysis web application.

Problem statement

Crime-scene images may contain important visual evidence such as bloodstains, knives and firearms. Manually locating these objects can be time-consuming. The project creates a web-based computer-vision workflow that accepts an image, runs object detection and returns dynamic predictions.

Architecture

User → React.js + Tailwind UI → image upload → Flask REST API → YOLOv8/object detection → predictions → React result interface

End-to-end approach

- 1 The user uploads a crime-scene image through the React interface.
- 2 The frontend sends the image to the Flask backend through a REST API.
- 3 The backend connects the request to the computer-vision inference workflow.
- 4 YOLOv8/object detection processes the image.
- 5 Supported targets include bloodstains, knives and firearms.
- 6 The prediction result is returned and displayed dynamically in the web application.

Implementation stack

- Frontend: React.js + Tailwind CSS.
- Backend/API: Python + Flask + REST APIs.
- ML: YOLOv8 / object detection.
- Domain: Computer Vision / image-based analysis.

Future roadmap

- Expand training data and supported crime-scene classes.
- Add formal evaluation reporting such as precision, recall and mAP when measured.
- Improve robustness for lighting, blur, camera angle and image-quality variation.
- Improve confidence-threshold configuration and bounding-box visualization.
- Add model versioning and reproducible deployment.
- Add secure image handling and audit logging if persistent image storage is introduced.

6. Project — SilentSOS

Voice-based intelligent safety system / discreet mobile safety application disguised as a calculator.

Problem statement

In a threatening situation, opening a conventional safety application or manually composing an emergency message may be difficult or unsafe. SilentSOS explores a discreet workflow in which the mobile application can listen for potential distress, classify short audio samples and trigger an emergency response.

Architecture

React Native + Expo → audio capture → FFmpeg → Librosa features → Python/Flask TensorFlow inference → danger confidence → Node.js/Express emergency workflow → MongoDB / trusted contacts / location

Audio ML pipeline

- 1 Smart listening records a short clip approximately every five seconds when activated.
- 2 Audio is converted to 16 kHz mono WAV when required.
- 3 Librosa extracts 40 MFCC features, pitch, zero-crossing rate and RMS energy.
- 4 These produce a 43-dimensional feature representation.
- 5 The TensorFlow stress model returns danger confidence.
- 6 A configured confidence above 0.6 is treated as a potential danger event.

Deep-learning model

- Bidirectional LSTM architecture.
- Dropout and dense layers.
- Class weighting for imbalanced data.
- Early stopping during training.
- Exported trained model: stress_model.h5.

Emergency workflow

- Capture emergency audio.
- Obtain GPS coordinates.
- Send SOS information to trusted contacts with Google Maps location and audio attachment.
- Node.js/Express handles emergency processing.
- MongoDB stores alert information and supports geospatial NGO lookup.
- Socket.IO supports real-time location updates.

Future roadmap

- Move validated inference closer to the device to reduce latency and improve privacy.
- Replace placeholder client-side ML helpers with validated TensorFlow.js inference where appropriate.
- Combine vocal stress, keyword detection, motion and contextual signals.
- Improve background execution on Android and iOS.
- Use secure production APIs and HTTPS.
- Add production SMS/emergency-service integrations if implemented.
- Build a trusted-contact/emergency dashboard for active incidents.

7. Project — FinGrow

Multi-service fintech platform combining P2P lending, KYC, financial analysis and market insights.

Problem statement

Financial workflows are often split across separate services for identity, lending, planning and market information. FinGrow brings these capabilities into one full-stack platform with a separate Python financial-analysis service.

System architecture

React + Vite → React Router / Context → Node.js + Express REST APIs → MongoDB + Mongoose React financial dashboard → Flask financial-analysis service → savings calculation + Scikit-learn Linear Regression + Gemini insights Additional integrations → TradingView / market visualization + Chatbase + ethers.js / MetaMask

Core workflow

- 1 User registration/login is handled through the web application and backend authentication.
- 2 JWT/cookies and bcrypt support authentication and protected access.
- 3 KYC information can cover basic details, PAN, Aadhaar and bank-verification information.
- 4 Lenders can create loan offers with amount and interest information.
- 5 Borrowers can retrieve matching offers through backend APIs.
- 6 Loan and transaction models/APIs provide the foundation for loan-related activity.
- 7 Financial inputs are sent to the Flask service for savings analysis and Linear Regression-based savings estimation.
- 8 Gemini provides financial/investment insights in the financial-analysis workflow.
- 9 TradingView, Chatbase and MetaMask/ethers.js add market, assistant and wallet functionality.

Important current-state boundary

The knowledge base should not claim that FinGrow already contains a fully automated production lender → borrower → disbursement → repayment lifecycle. The current architecture includes loan and transaction foundations; a more complete repayment and investor workflow is part of the planned roadmap.

Future roadmap

- Complete lender-borrower transaction lifecycle.
- Production repayment system and status updates.
- Dedicated credit-risk model with a proper dataset and evaluation pipeline.
- Investor/lender dashboards.
- Stronger KYC integrations.
- Notifications.
- Better ML data collection, evaluation and monitoring.

8. Project — FitGenius AI

Personalized diet and workout recommendation system.

Problem statement

Generic fitness plans may not account for an individual's body metrics, goals and preferences. FitGenius AI uses user-specific inputs and an ML workflow to produce a tailored plan.

Architecture

React frontend → Flask ML API → data preprocessing / feature engineering → Scikit-learn model → prediction → personalized diet and workout recommendation

Approach

- 1 Collect user body metrics, goals and preferences.
- 2 Preprocess the input data.
- 3 Perform feature engineering.
- 4 Run the Scikit-learn model.
- 5 Transform the prediction into a personalized recommendation.
- 6 Return the recommendation to the React interface through Flask.

Future roadmap

- Expand and validate the training dataset.
- Add stronger validation and model metrics.
- Improve personalization and constraints.
- Add recommendation history and feedback.
- Introduce monitoring for recommendation quality and drift.

9. Project — CodePilot AI

Multi-agent RAG coding assistant for codebase analysis and natural-language code queries.

Problem statement

Large codebases are difficult to understand through manual browsing. Developers need a way to ask natural-language questions about files, functions, architecture and dependencies while keeping answers grounded in the repository.

Pipeline architecture

Repository ingestion → parsing → dependency graph → chunking + embeddings → guardrails → intent router → retrieval / overview / casual path → answer agent → validation → cache → response

Query routing

- Specific: retrieval is used to locate relevant code/context.
- Broad: project-level overview context can answer the question without unnecessary deep retrieval.
- Casual: retrieval can be bypassed.
- This avoids paying the cost of full retrieval for every question.

Detailed components

- Input guardrails and LLM guardrail layer.
- Answer cache.
- Intent Router.
- Retrieval branch with retrieval guard and expanded-retry behavior.
- Project Overview branch.
- Casual response branch.
- Answer Agent.
- Answer validation.

Implementation details represented in the portfolio

- FastAPI backend.
- LangGraph multi-agent orchestration.
- React + Vite frontend.
- Project history, file browsing and code display.
- Endpoints: /ask, /ask/stream (SSE) and /ingest.
- The resume/source also describes Gemini embeddings, Supabase, HNSW indexing, SHA-256 chunk IDs, PostgreSQL pooling, JWT authentication and Docker. Preserve those source-specific details when asked about the exact implementation.

Future roadmap

- More dependency-aware retrieval.
- Improved cross-file reasoning.
- Better evaluation datasets for code-grounded answers.
- Incremental ingestion and cache optimization.
- Stronger security and sandboxing for code execution.

10. Project — AgentForge

Autonomous AI Website Builder — natural language in, a structured and tested web application out.

Problem statement

A single LLM prompt can generate plausible code, but it does not reliably manage requirements, frontend/backend/database contracts, build failures or deployment. AgentForge turns app generation into a coordinated engineering pipeline with specialized planning and generation agents.

Multi-agent architecture

User Idea → Requirement Analyzer → Master Planner → Frontend Planner / Backend Planner / Database Planner → Generator Agents → Docker → Build/Test → Deploy

Agent responsibilities

- Requirement Analyzer — converts free text into structured, testable requirements.
- Master Planner — coordinates the overall project plan.
- Frontend Planner — designs frontend pages, components and implementation structure.
- Backend Planner — plans APIs, services and business logic.
- Database Planner — designs models and persistence.
- Generator Agents — write implementation code.
- Docker — provides a reproducible build/test environment.
- Build/Test — validates the generated project and can feed failures back into the workflow.
- Deploy — ships a tested application.

Key engineering challenge

The major challenge is coordination: specialized agents can work on separate concerns, but they must agree on shared contracts such as routes, schemas, types and dependencies. Generated code also has to be buildable rather than merely plausible.

Evaluation

- Does the generated project build?
- Do its available tests pass?
- Can the generated application deploy without manual intervention?
- Do individual agents produce the expected structured state and artifacts?

Future roadmap

- Feedback from deployed-app failures back into planning.
- Support extending an existing generated application.
- Pluggable planners for additional technology stacks.
- Better agent synchronization and parallel execution.
- Stronger dependency recovery and regression testing.

11. Project — Self-Healing Debugger

Autonomous debugging and self-recovery system for Python projects.

Problem statement

Traditional debugging requires manually reading a traceback, locating source code, diagnosing the root cause, creating a patch, running tests and checking regressions. A generated fix cannot simply be trusted. This project creates a controlled repair pipeline where AI-generated patches are analyzed, validated, sandbox-tested and risk-classified before application.

Architecture

Run project → detect error → parse traceback → AST source context → root-cause diagnosis → related-code retrieval → patch generation → static validation → sandbox test → regression analysis → risk classification / human approval → backup → apply

Detailed workflow

- 1 Run the Python entry file and capture stdout, stderr and exit code.
- 2 Parse the traceback into error type, message, file, line, function and stack trace.
- 3 Use Python AST to locate the failing function and its called functions.
- 4 Use Gemini 2.5 Flash to diagnose the root cause.
- 5 Retrieve related functions using bounded AST-based traversal with cycle protection.
- 6 Generate a complete replacement for the failing function while preserving its name and parameters.
- 7 Validate syntax, signature preservation and configured dangerous operations.
- 8 Apply the candidate patch only inside a temporary copied project.
- 9 Run project execution and pytest when available.
- 10 Use regression analysis to decide whether the patch should pass, retry or fail.
- 11 Classify risk and pause high-risk changes for explicit human approval.
- 12 Create a .bak backup before writing an approved/low-risk fix to the original source.

Agents / components

- Log Monitor Agent
- Root Cause Analyzer Agent
- Fix Generator Agent
- Static Analysis Agent
- Sandbox Test Agent
- Regression Checker Agent
- Risk Classifier
- Rollback / backup mechanism

High-risk change policy

Changes involving authentication, payment, login, password, token or security-sensitive files are treated as high risk and require human approval before the original source is changed.

Future roadmap

- Support additional programming languages.
- Improve patch verification and mutation testing.
- Strengthen sandbox isolation and resource limits.
- Build persistent failure/repair memory.

- Automate rollback after failed health checks.
- Create dedicated evaluation sets for patch correctness and safety.

12. Portfolio Website — The System That Hosts the Knowledge

React + Vite portfolio with Tailwind styling, custom visual scenes and a demo assistant designed to be replaced by a real RAG backend.

Architecture

React application → reusable UI components → structured data modules → page-level case studies / background / skills / journey / contact → scripted assistant interface

Portfolio structure

- Home — professional introduction, AI/agent positioning, featured projects and system snapshot.
- About — engineering philosophy, build stages and current direction.
- Projects — project catalog grouped into Agentic AI, Generative AI, ML/Deep Learning and AI + Full Stack.
- Skills — detailed technology areas, contribution types, roles and proficiency framing.
- Journey — timeline from full-stack development through autonomous AI systems.
- Background — education, practical experience, training, achievements and current focus.
- Contact — email, phone, social links, resume and AI assistant entry point.
- Case Study pages — project overview, problem, workflow, implementation, challenges, evaluation and future roadmap.

Visual / engineering details

- React 18 + Vite.
- Tailwind CSS.
- Custom canvas-based 3D scenes rather than a Three.js dependency.
- Reusable components such as Navbar, Hero, SectionHeading, Reveal, PageCTA, GraphScene and assistant components.
- Structured data modules keep portfolio content separate from UI components.
- Project architecture is represented through graph data such as AGENTFORGE, DEBUGGER, FINGROW, FITGENIUS and CODEPILOT.

Current assistant status

The portfolio source describes the assistant as a scripted/demo assistant that is structured so it can be swapped for a real RAG/agent backend. The new RAG knowledge base is therefore intended to become the grounding source for that next stage.

13. Achievements, Problem Solving & Professional Direction

Achievement	Best Startup Award — Code Crafter 2.0
Leadership	Coordinator — Udaan, CT University
Problem solving	Portfolio displays 950+ LeetCode problems; resume/source separately states 1200+ DSA problems across LeetCode, GeeksforGeeks and Coding Ninjas
Core problem types	Arrays, Strings, Trees, Graphs, Dynamic Programming, Binary Search, Greedy

Important DSA conflict rule

Do not answer with a single DSA number if the user asks about exact counts without source context. The uploaded sources contain different figures. The assistant should identify the source: portfolio shows 950+ LeetCode, while the resume states 1200+ DSA problems across LeetCode, GeeksforGeeks and Coding Ninjas.

Current focus

- AI / Machine Learning
- Generative AI
- Agentic AI
- LangGraph
- MCP
- RAG
- AI-powered software systems
- AI evaluation and production reliability

Current roles of interest represented in the portfolio

- AI / ML Intern — ML models, data preprocessing, model evaluation and Python.
- AI Engineering Intern — LLM applications, RAG, agents and LangGraph.
- Full-Stack AI — React, FastAPI/Node, databases and AI integration.

14. Portfolio-Wide Future Roadmap

- Deepen tool-using agents, LangGraph state machines and memory.
- Build repeatable evaluations for agent and RAG quality.
- Improve observability, deployment and reliability for AI services.
- Coordinate specialized agents on larger, longer workflows.
- Move toward autonomous software that can plan, execute, evaluate and recover.
- Add stronger production security, secret management and environment isolation.
- Expand automated testing across frontend, backend, ML and agent workflows.
- Introduce model/version tracking and regression testing for AI components.

15. RAG Assistant — Exact Answering Rules

- Answer only from retrieved knowledge-base content and approved portfolio/resume context.
- Do not use outside internet knowledge to fill project-specific gaps.
- Never invent metrics, project features, technologies, dates, certifications, employers or responsibilities.
- For the Solitair Infosys internship, always use Crime Scene Detection as the project.
- When an implementation detail is explicitly available, preserve the terminology used in the source.
- Clearly distinguish current/completed features from planned/future improvements.
- If the answer is not available, say: "I don't have that information in Pourush's portfolio/project knowledge base."
- If sources conflict, state both values and identify the source instead of silently merging them.
- Do not claim that every listed skill is used in every project.
- Do not convert portfolio marketing language into unsupported technical claims.

16. Recommended chunking strategy

- Create a summary chunk for each project.
- Create separate chunks for problem statement, architecture, workflow, implementation, challenges, evaluation and future roadmap.
- Keep architecture diagrams/text with the project name in the same retrieval context.
- Use metadata such as project_name, section, model, technology, component, source_type and version.
- Create a dedicated contact/profile chunk containing email, phone and professional URLs.
- Create a dedicated source-conflict chunk for internship and DSA discrepancies.

17. High-value questions this knowledge base should answer

- Who is Pourush Kashyap?
- What is Pourush's educational background?
- How can I contact Pourush?
- What is his LinkedIn / GitHub / LeetCode?
- What was his Solitair Infosys internship project?
- Explain the architecture of Crime Scene Detection.
- How does SilentSOS process audio?
- Explain FinGrow end to end.

- What is CodePilot AI and how does its RAG pipeline work?
- Explain AgentForge's multi-agent architecture.
- How does the Self-Healing Debugger safely apply fixes?
- What technologies does Pourush use?
- What is Pourush currently learning?
- What are his future project plans?
- What is the difference between his current features and future roadmap?

This document is intentionally more detailed than a normal portfolio summary. It gives a RAG system enough context to answer both short recruiter questions and deeper technical questions about architecture and implementation. The contact section is intentionally explicit so the assistant can act as a portfolio interface and route interested visitors toward Pourush's email and professional profiles.

End of Deep RAG Knowledge Base

Pourush Kashyap — Project Technology Stack Addendum

This addendum makes project-specific technology stacks explicit for retrieval. It should be ingested together with the existing Deep RAG Knowledge Base. The stacks below are project-specific and should not be generalized to every project.

Important: Some technologies were already present in the original knowledge base, while this addendum makes the mapping explicit so questions such as "What technologies does AgentForge use?" can be answered from project context.

Crime Scene Detection

Frontend: React.js, Tailwind CSS
Backend/API: Python, Flask, REST APIs
ML/CV: YOLOv8, object detection
Domain: Computer Vision / image-based analysis

SilentSOS

Mobile: React Native, Expo
Audio processing: FFmpeg, Librosa
ML/DL: TensorFlow, Bidirectional LSTM
ML API: Python, Flask
Backend/emergency workflow: Node.js, Express.js
Database: MongoDB
Realtime: Socket.IO

FinGrow

Frontend: React, Vite, React Router, Context
Backend: Node.js, Express.js, REST APIs
Database: MongoDB, Mongoose
Financial-analysis service: Python, Flask
ML: Scikit-learn Linear Regression
GenAI: Gemini
Integrations: TradingView, Chatbase, ethers.js / MetaMask
Authentication: JWT/cookies, bcrypt

FitGenius AI

Frontend: React
ML API: Flask
ML: Scikit-learn
Pipeline: preprocessing, feature engineering, model prediction
Purpose: personalized diet and workout recommendations

CodePilot AI

Backend: FastAPI
Orchestration: LangGraph
Frontend: React, Vite
RAG: embeddings, retrieval, vector search
Embeddings: Gemini embeddings
Vector/search infrastructure: Supabase, HNSW

Database/auth/infra: PostgreSQL pooling, JWT, Docker
Identifiers: SHA-256 chunk IDs

AgentForge

Orchestration: LangGraph
Backend/API: FastAPI
Frontend: React, TypeScript, Tailwind CSS
Agent architecture: Requirement Analyzer, Master Planner, Frontend/Backend/Database Planners, Generator
Agents
Build/test/deployment: Docker
Workflow: planning → generation → build/test → deployment

Self-Healing Debugger

Orchestration: LangGraph
Backend/API: FastAPI
Frontend: React
LLM: Gemini 2.5 Flash
Code analysis: Python AST
Testing: pytest
Execution/recovery: sandbox testing, regression analysis, risk classification, backup/rollback

Portfolio Website

Frontend: React 18, Vite, Tailwind CSS
Visuals: custom canvas-based 3D scenes
Architecture: reusable React components, structured data modules
Assistant: scripted/demo assistant intended to be replaced by the real RAG backend

Retrieval guidance

For project-specific technology questions, prefer the technology stack section for the named project. Do not infer that a technology listed in the global Technical Skill Map is used in every project. When a technology is described as future/planned in the main knowledge base, preserve that status.

PROJECT TECHNOLOGY STACK

Corrected project-specific technology reference. This section reflects the project technologies provided for the portfolio and the user's correction that SilentSOS does not use TypeScript.

1. Crime Scene Detection

Area	Technology / Tools
Frontend	React.js, Tailwind CSS
Backend / API	Python, Flask, REST APIs
ML / Computer Vision	YOLOv8, object detection
Domain	Computer Vision / image-based analysis

2. SilentSOS

Area	Technology / Tools
Mobile	React Native, Expo
Language	JavaScript
Audio Processing	FFmpeg, Librosa
ML / Deep Learning	TensorFlow, Bidirectional LSTM
ML API	Python, Flask
Emergency Backend	Node.js, Express.js
Database	MongoDB
Realtime	Socket.IO

3. FinGrow

Area	Technology / Tools
Frontend	React, Vite, React Router, Context, JavaScript
Backend / API	Node.js, Express.js, REST APIs
Database	MongoDB, Mongoose
Financial Analysis	Python, Flask
Machine Learning	Scikit-learn, Linear Regression
GenAI	Gemini
Authentication	JWT/cookies, bcrypt
Integrations	TradingView, Chatbase, ethers.js / MetaMask

4. FitGenius AI

Area	Technology / Tools
Frontend	React, JavaScript
ML API	Flask
Machine Learning	Scikit-learn
Pipeline	Preprocessing, feature engineering, model prediction
Purpose	Personalized diet and workout recommendations

PROJECT TECHNOLOGY STACK — CONTINUED

5. CodePilot AI

Area	Technology / Tools
Backend	FastAPI
Orchestration	LangGraph
Frontend	React, Vite, JavaScript
RAG	Embeddings, retrieval, vector search
Embeddings	Gemini embeddings
Vector / Search	Supabase, HNSW
Database / Auth / Infra	PostgreSQL pooling, JWT, Docker
Identifiers	SHA-256 chunk IDs

6. AgentForge

Area	Technology / Tools
Orchestration	LangGraph
Backend / API	FastAPI
Frontend	React, JavaScript, Tailwind CSS
Agent Architecture	Requirement Analyzer, Master Planner, Frontend Planner, Backend Planner, Database Planner, Generator Agents
Build / Test / Deployment	Docker
Workflow	Planning → generation → build/test → deployment

7. Self-Healing Debugger

Area	Technology / Tools
Orchestration	LangGraph
Backend / API	FastAPI
Frontend	React, JavaScript
LLM	Gemini 2.5 Flash
Code Analysis	Python AST
Testing	pytest
Execution / Recovery	Sandbox testing, regression analysis, risk classification, backup/rollback

8. Portfolio Website

Area	Technology / Tools
Frontend	React 18, Vite, Tailwind CSS, JavaScript
Visuals	Custom canvas-based 3D scenes
Architecture	Reusable React components, structured data modules
Assistant	Scripted/demo assistant intended to be replaced by the real RAG backend

Correction applied: SilentSOS is listed with JavaScript and no TypeScript. The other project frontend stacks use JavaScript where applicable; AgentForge is also listed with JavaScript.